

Contents

Complete SQL Notes — Basic to Advanced	1
1. Introduction to SQL	1
2. Database & Table Basics	2
3. Data Types (common ones)	2
4. DDL — Data Definition Language	2
5. Constraints (Rules on Cells/Columns)	3
6. DML — Data Manipulation Language	4
7. DQL — SELECT and Filtering	4
8. Aggregate Functions & Grouping	5
9. Joins (Combining Tables)	5
10. Subqueries	7
11. Common Table Expressions (CTE)	7
12. Set Operations	8
13. String, Date, and Numeric Functions	8
14. Views	9
15. Indexes	9
16. Transactions & ACID	9
17. Window Functions (Advanced)	10
18. Stored Procedures & Functions	10
19. Triggers	10
20. Normalization (Database Design)	11
21. Query Execution Order (how SQL actually processes a SELECT)	11
22. Performance & Optimization Tips (Advanced)	11
Quick Reference — SELECT Statement Full Syntax	12

Complete SQL Notes — Basic to Advanced

1. Introduction to SQL

SQL (Structured Query Language) is used to communicate with relational databases (RDBMS) like MySQL, PostgreSQL, SQL Server, Oracle, SQLite.

Categories of SQL Commands

Category	Full Form	Purpose	Commands
DDL	Data Definition Language	Defines/modifies structure	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	Modifies data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieves data	SELECT
DCL	Data Control Language	Controls access	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions	COMMIT, ROLLBACK, SAVEPOINT

2. Database & Table Basics

- **Database:** A container that holds related tables.
- **Table:** A grid of rows (records) and columns (fields/attributes).
- **Schema:** The structure/blueprint of a database (tables, columns, relationships).

CREATE DATABASE company;

USE company;

- CREATE DATABASE → makes a new database.
 - USE → switches the active database for subsequent commands.
-

3. Data Types (common ones)

Type	What it stores
INT / INTEGER	Whole numbers
DECIMAL(p,s) / NUMERIC	Exact decimal numbers (p = total digits, s = digits after decimal)
FLOAT / DOUBLE	Approximate decimal numbers
VARCHAR(n)	Variable-length text, max n characters
CHAR(n)	Fixed-length text, always n characters
TEXT	Large text blocks
DATE	Date only (YYYY-MM-DD)
DATETIME / TIMESTAMP	Date + time
BOOLEAN	TRUE/FALSE
BLOB	Binary data (images, files)

4. DDL — Data Definition Language

4.1 CREATE TABLE

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50),  
    salary DECIMAL(10,2) DEFAULT 0,  
    dept_id INT,  
    hire_date DATE  
);
```

What each part does (cell/column-level): - emp_id INT PRIMARY KEY → the column that uniquely identifies each row (no duplicates, no NULLs). - first_name VARCHAR(50) NOT NULL → this cell can never be left empty for any row. - salary DECIMAL(10,2) DEFAULT 0 → if no value is given during insert, the cell auto-fills with 0. - dept_id INT → will later be linked (FOREIGN KEY) to a departments table. - hire_date DATE → stores only the date, no time.

4.2 ALTER TABLE

```
ALTER TABLE employees ADD COLUMN email VARCHAR(100);
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2);
ALTER TABLE employees DROP COLUMN email;
ALTER TABLE employees RENAME COLUMN first_name TO fname;
```

- ADD COLUMN → inserts a brand-new empty cell (NULL) for every existing row.
- MODIFY COLUMN → changes the data type/size of an existing column (all cells keep their data if compatible).
- DROP COLUMN → deletes that column and its data from every row.
- RENAME COLUMN → just changes the column name, data unaffected.

4.3 DROP vs TRUNCATE vs DELETE

Command	Removes	Structure kept?	Rollback possible?	Speed
DROP TABLE	Table + data + structure	No	No (usually)	N/A
TRUNCATE TABLE	All rows	Yes	No (usually)	Very fast
DELETE FROM table	Rows (optionally filtered)	Yes	Yes (with WHERE, transactional)	Slower

5. Constraints (Rules on Cells/Columns)

Constraint	What it enforces on a cell
PRIMARY KEY	Cell(s) must be unique + never NULL; identifies the row
FOREIGN KEY	Cell value must exist in another table's referenced column
UNIQUE	No two rows can have the same value in this cell
NOT NULL	Cell cannot be left empty
CHECK	Cell value must satisfy a condition, e.g. CHECK (salary > 0)
DEFAULT	If no value given, cell auto-fills with a default
AUTO_INCREMENT / IDENTITY	Cell auto-generates the next number automatically

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY AUTO_INCREMENT,  
    dept_name VARCHAR(50) UNIQUE NOT NULL  
);
```

```
ALTER TABLE employees  
ADD CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id);
```

- AUTO_INCREMENT → every new row's dept_id cell auto-fills as 1, 2, 3... without you typing it.

- FOREIGN KEY (dept_id) REFERENCES departments(dept_id) → every value put into employees.dept_id must already exist as a dept_id in departments; prevents “orphan” rows.

6. DML — Data Manipulation Language

6.1 INSERT

```
INSERT INTO employees (emp_id, first_name, last_name, salary, dept_id, hire_date)
VALUES (1, 'Aman', 'Verma', 50000, 1, '2023-01-15');
```

```
INSERT INTO employees (emp_id, first_name, salary)
VALUES (2, 'Riya', 45000);
```

- Only listed columns get real values; unlisted columns become NULL (or their DEFAULT).

6.2 UPDATE

```
UPDATE employees
SET salary = salary + 5000
WHERE dept_id = 1;
```

- SET → tells SQL which cell(s) to change and to what new value.
- WHERE → limits the update to only the rows that match; **omitting WHERE updates every row's cell** — a very common mistake.

6.3 DELETE

```
DELETE FROM employees WHERE emp_id = 2;
```

- Removes entire row(s) matching the condition. Without WHERE, deletes all rows (structure remains).

7. DQL — SELECT and Filtering

7.1 Basic SELECT

```
SELECT first_name, salary FROM employees;
SELECT * FROM employees; -- * means "all columns"
```

7.2 WHERE (row filter)

```
SELECT * FROM employees WHERE salary > 40000;
SELECT * FROM employees WHERE dept_id = 1 AND salary > 40000;
SELECT * FROM employees WHERE first_name LIKE 'A%'; -- starts with A
SELECT * FROM employees WHERE dept_id IN (1,2,3);
SELECT * FROM employees WHERE salary BETWEEN 30000 AND 60000;
SELECT * FROM employees WHERE last_name IS NULL;
```

Operator	Meaning
=, !=/<>, >, <, >=, <=	Comparison
AND, OR, NOT	Logical combination

Operator	Meaning
LIKE	Pattern match (% = any chars, _ = one char)
IN (...)	Value matches any in a list
BETWEEN a AND b	Value falls in a range (inclusive)
IS NULL / IS NOT NULL	Checks for empty cells

7.3 ORDER BY (sorting)

```
SELECT * FROM employees ORDER BY salary DESC;           -- highest first
SELECT * FROM employees ORDER BY dept_id, salary DESC; -- multi-level sort
```

7.4 LIMIT / OFFSET (pagination)

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;           -- top 5 rows
SELECT * FROM employees ORDER BY salary DESC LIMIT 5 OFFSET 10; -- skip 10, then take 5
```

8. Aggregate Functions & Grouping

Function	What it computes
COUNT()	Number of rows / non-NULL values
SUM()	Total of a numeric column
AVG()	Average of a numeric column
MIN() / MAX()	Smallest / largest value

```
SELECT dept_id, COUNT(*) AS total_employees, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 40000
ORDER BY avg_salary DESC;
```

- GROUP BY dept_id → collapses all rows sharing the same dept_id into one summary row per department.
- COUNT(*) cell → counts how many employee rows fell into that group.
- AVG(salary) cell → the mean of the salary column *within that group only*.
- HAVING → filters *groups* after aggregation (unlike WHERE, which filters rows *before* aggregation).

WHERE vs HAVING	WHERE	HAVING	Filters	Individual rows	Grouped/aggregated results	Runs	Before GROUP BY	After GROUP BY	Can use aggregate functions?
			Yes	Yes	No	Yes			

9. Joins (Combining Tables)

Sample tables: employees(emp_id, name, dept_id) and departments(dept_id, dept_name).

9.1 INNER JOIN

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

- Returns only rows where the dept_id cell matches in **both** tables. Employees with no matching department (or NULL dept_id) are excluded.

9.2 LEFT JOIN (LEFT OUTER JOIN)

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

- Keeps **every row from employees** (left table), even if there's no matching department — in that case the dept_name cell shows NULL.

9.3 RIGHT JOIN

- Same idea, but keeps every row from the **right** table (departments), filling NULL where no employee matches.

9.4 FULL OUTER JOIN

- Keeps all rows from **both** tables; unmatched cells on either side become NULL.

9.5 CROSS JOIN

```
SELECT e.name, d.dept_name FROM employees e CROSS JOIN departments d;
```

- Produces every possible combination of rows (Cartesian product) — no matching condition.

9.6 SELF JOIN

```
SELECT a.name AS employee, b.name AS manager
FROM employees a
JOIN employees b ON a.manager_id = b.emp_id;
```

- A table joined to itself — useful for hierarchical data like employee → manager.

Visual summary

Join Type	Rows Returned
INNER JOIN	Only matching rows in both
LEFT JOIN	All left + matched right (NULL if none)
RIGHT JOIN	All right + matched left (NULL if none)
FULL JOIN	All rows from both, matched where possible
CROSS JOIN	Every combination
SELF JOIN	Table matched against itself

10. Subqueries

10.1 Scalar/Column Subquery

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

- The inner query runs first, returns one value (the average), then the outer query filters rows above it.

10.2 Subquery with IN

```
SELECT name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Delhi');
```

10.3 Correlated Subquery

```
SELECT name, salary FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);
```

- Inner query re-runs **for every row** of the outer query, referencing the outer row's dept_id each time — compares each employee's salary to their *own department's* average.

10.4 Subquery in FROM (derived table)

```
SELECT dept_id, avg_salary
FROM (SELECT dept_id, AVG(salary) AS avg_salary FROM employees GROUP BY dept_id) AS dept_avg
WHERE avg_salary > 40000;
```

11. Common Table Expressions (CTE)

```
WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
)
SELECT e.name, e.salary, d.avg_salary
FROM employees e
JOIN dept_avg d ON e.dept_id = d.dept_id;
```

- WITH ... AS (...) → defines a temporary named result set (dept_avg) that exists only for this query, making complex queries more readable than nested subqueries.

Recursive CTE (e.g., organization hierarchy)

```
WITH RECURSIVE org_chart AS (
    SELECT emp_id, name, manager_id, 1 AS level
    FROM employees WHERE manager_id IS NULL           -- anchor: the top boss
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.level + 1
    FROM employees e
    JOIN org_chart o ON e.manager_id = o.emp_id       -- recursive: each level below
```

```
)
SELECT * FROM org_chart;
```

12. Set Operations

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers;      -- combines + removes duplicates
```

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers;      -- combines, keeps duplicates
```

```
SELECT city FROM customers
INTERSECT
SELECT city FROM suppliers;      -- only cities in BOTH
```

```
SELECT city FROM customers
EXCEPT
SELECT city FROM suppliers;      -- MINUS in Oracle
                                   -- cities in customers but NOT suppliers
```

Rule: all combined SELECTs must have the **same number of columns with compatible data types**.

13. String, Date, and Numeric Functions

String

Function	Purpose	Example
CONCAT()	Join strings	CONCAT(first_name, ' ', last_name)
UPPER() / LOWER()	Change case	UPPER(name)
LENGTH()	Character count	LENGTH(name)
SUBSTRING(str, start, len)	Extract part of a string	SUBSTRING(name, 1, 3)
TRIM()	Remove leading/trailing spaces	TRIM(name)
REPLACE(str, old, new)	Replace text	REPLACE(name, 'a', '@')

Date

Function	Purpose
NOW() / CURRENT_TIMESTAMP	Current date & time
CURDATE()	Current date
DATEDIFF(d1, d2)	Days between two dates
DATE_ADD(date, INTERVAL n DAY)	Add time to a date
EXTRACT(YEAR FROM date)	Pull out year/month/day

Numeric

Function	Purpose
ROUND(n, d)	Round to d decimal places
CEIL() / FLOOR()	Round up / down
ABS()	Absolute value
MOD(a,b)	Remainder

14. Views

```
CREATE VIEW high_earners AS
SELECT name, salary, dept_id FROM employees WHERE salary > 60000;

SELECT * FROM high_earners;    -- query it just like a table
```

- A **view** is a saved, virtual table built from a query — it doesn't store data itself, it re-runs the underlying query each time you SELECT from it. Useful for simplifying complex joins and restricting column/row access.

```
DROP VIEW high_earners;
CREATE OR REPLACE VIEW high_earners AS ...    -- update a view's definition
```

15. Indexes

```
CREATE INDEX idx_salary ON employees(salary);
DROP INDEX idx_salary ON employees;
```

- An index is a lookup structure (like a book's index page) built on a column so the database can find matching rows fast instead of scanning every row (a "full table scan").
 - Speeds up WHERE, JOIN, and ORDER BY on that column.
 - Trade-off: indexes speed up reads but slightly slow down INSERT/UPDATE/DELETE (since the index must also be updated) and use extra storage.
 - PRIMARY KEY and UNIQUE columns get an index automatically in most databases.
-

16. Transactions & ACID

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 1000 WHERE acc_id = 1;
UPDATE accounts SET balance = balance + 1000 WHERE acc_id = 2;

COMMIT;    -- makes changes permanent
-- or
ROLLBACK;  -- undoes everything since START TRANSACTION
```

- SAVEPOINT sp1; → marks a point you can roll back to without undoing the whole transaction (ROLLBACK TO sp1;).

ACID properties: | Property | Meaning | |—|—| | Atomicity | All statements in a transaction succeed, or none do | | Consistency | Database moves from one valid state to another | | Isolation | Concurrent transactions don't interfere with each other | | Durability | Once committed, changes survive even a crash |

17. Window Functions (Advanced)

Unlike GROUP BY, window functions calculate across a set of rows **without collapsing them** — every original row stays visible.

```
SELECT name, dept_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num,
       SUM(salary) OVER (PARTITION BY dept_id) AS dept_total,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees;
```

- PARTITION BY dept_id → splits rows into groups (like GROUP BY), but each row is still returned individually.
 - ORDER BY salary DESC (inside OVER) → decides the ranking order within each partition.
 - RANK() → gives the same rank to ties, then skips numbers (1,1,3...).
 - DENSE_RANK() → same as RANK but doesn't skip numbers after ties (1,1,2...).
 - ROW_NUMBER() → always gives a unique sequential number, even for ties.
 - LAG(col) / LEAD(col) → gets the value from the previous/next row in the partition — handy for comparing to prior period.
-

18. Stored Procedures & Functions

```
DELIMITER //
CREATE PROCEDURE GetHighEarners(IN min_salary DECIMAL(10,2))
BEGIN
    SELECT name, salary FROM employees WHERE salary > min_salary;
END //
DELIMITER ;
```

```
CALL GetHighEarners(50000);
```

- A **stored procedure** is a saved block of SQL logic (can include loops, conditionals, multiple statements) that you call by name instead of retyping the query.
 - A **function** is similar but must RETURN a single value and can be used inside a SELECT like a normal function.
-

19. Triggers

```
CREATE TRIGGER before_salary_update
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < OLD.salary THEN
        SET NEW.salary = OLD.salary; -- prevent salary decrease
```

END IF;
END;

- A trigger auto-runs a block of logic whenever an INSERT, UPDATE, or DELETE happens on a table.
- OLD.column → the cell's value before the change. NEW.column → the cell's value being written.

20. Normalization (Database Design)

Normal Form	Rule
1NF	Each cell holds a single (atomic) value — no lists/multiple values in one cell
2NF	1NF + every non-key column depends on the <i>whole</i> primary key (relevant for composite keys)
3NF	2NF + no non-key column depends on another non-key column (no transitive dependency)
BCNF	Stricter version of 3NF — every determinant must be a candidate key

Goal: reduce data redundancy and avoid update/insert/delete anomalies by splitting data into related tables connected via foreign keys.

21. Query Execution Order (how SQL actually processes a SELECT)

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

This is why you can't reference a SELECT alias inside WHERE, but you often can in ORDER BY — WHERE runs before SELECT even builds the alias.

22. Performance & Optimization Tips (Advanced)

- Use indexes on columns frequently used in WHERE, JOIN, ORDER BY.
- Avoid SELECT * in production code — fetch only needed columns.
- Use EXPLAIN (or EXPLAIN ANALYZE) before a query to see how the database plans to execute it (table scan vs index scan).
- Prefer EXISTS over IN for large subqueries — it can stop as soon as one match is found.
- Avoid functions on indexed columns in WHERE (e.g. WHERE YEAR(hire_date)=2024 disables the index; use a date range instead).
- Normalize for data integrity, but selectively denormalize for read-heavy reporting workloads.
- Batch large INSERT/UPDATE operations instead of row-by-row loops.

Quick Reference — SELECT Statement Full Syntax

```
SELECT [DISTINCT] column1, column2, AGG_FUNC(column3)
FROM table1
[JOIN table2 ON condition]
[WHERE row_condition]
[GROUP BY column1]
[HAVING group_condition]
[ORDER BY column1 [ASC|DESC]]
[LIMIT n [OFFSET m]];
```

End of notes.